

Music Genre Classification Using Multi-Branch Convolutional Neural Networks on the GTZAN Dataset

IS4402 Final Project Report

Tristan Tan

April 12, 2026

1 Introduction

Automatic music genre classification is a foundational task in Music Information Retrieval (MIR), underpinning content-based recommendation, mood estimation, and playlist generation in modern streaming platforms [1]. Genre serves as a high-level semantic descriptor that correlates strongly with listener preferences and informs downstream tasks including mood estimation and collaborative filtering. This project investigates whether deep learning over acoustic image representations can compete with and extend classical tabular-feature approaches on the widely used GTZAN benchmark [1, 2].

GTZAN comprises 1,000 audio tracks (30 s, 22,050 Hz mono WAV) uniformly distributed across 10 genres: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, and rock. The classification task presents several non-trivial challenges. Musical genres are not crisply defined: a blues-rock track may share acoustic properties with both blues and rock. Genres such as disco and hip-hop share rhythmic characteristics; classical and jazz share complex harmonic structure. The 1,000-track dataset is also small by deep-learning standards, placing strong constraints on model complexity and necessitating transfer learning. Finally, CNNs operate on 2D images rather than raw waveforms, so the choice of time-frequency representation materially affects what the network can learn [3].

Three progressively richer architectures are investigated. **Stage 1** trains a single ResNet-18 [4] on mel spectrograms, establishing a single-modality baseline. **Stage 2** fuses three parallel ResNet-18 branches processing mel, STFT, and chroma spectrograms [5], testing whether complementary representations provide additive discriminative signal. **Stage 3** augments the fusion model with a tabular MLP branch processing 54 hand-crafted MIR features [1], testing whether classical signal-processing statistics complement what the CNN branches learn from raw spectrograms. The primary metric is top-1 accuracy on a held-out test set constructed with strict track-level splitting to prevent segment-track leakage [2].

2 Data Preparation

2.1 Dataset Cleaning and Splitting

The GTZAN dataset [1] is publicly available on [Kaggle](#) and contains three components: (1) **genres original** –

10 genre folders each containing 100 WAV audio files of 30 seconds each (22,050 Hz, mono), totalling 1,000 tracks; (2) **images original** – pre-generated mel spectrogram PNG images for each audio file; and (3) **two CSV files** – a 30-second feature file (one row per track) and a 3-second feature file (one row per 3-second segment, 9,990 rows total), each containing the mean and variance of features such as MFCCs, spectral centroid, zero-crossing rate, RMS energy, and tempo. This project uses only the **WAV files** (genres original) and the **3-second CSV**; the pre-generated spectrogram images are discarded as we generate our own from the raw audio with consistent parameters across all three modalities.

Audio integrity is verified by loading all 1,000 WAV files with `librosa`. Cleaning proceeds in three steps: (1) `jazz.00054` is removed following documented evidence that it contains speech rather than music [2]; (2) tracks with `rms_mean` below 0.01 are flagged as near-silent; (3) tracks with ≥ 8 of 10 segments lying more than 3σ from their genre centroid in PCA feature space are flagged as anomalous (see Appendix B for the PCA visualisation). In total, 12 tracks are removed, yielding a cleaned dataset of **988 tracks** and **9,870 three-second segments**.

A critical design decision is splitting at the **track level**, not the segment level. Since the 10 three-second segments of a 30-second track are acoustically near-identical, a random segment-level split leaks information across the train/test boundary, artificially inflating accuracy [2]. Tracks are assigned to splits in a stratified 80/10/10 ratio, yielding 790/99/99 tracks (7,890/990/990 segments) in train/val/test. All spectrogram images and tabular feature rows are assigned to the same split as their parent track, ensuring zero cross-contamination.

2.2 Spectrogram Generation

Each 3-second segment is pre-converted to three 128×128 RGB spectrograms and saved to disk. All three representations use parameters $N_{\text{fft}} = 2048$, hop length $H = 512$, $f_s = 22,050$ Hz, producing approximately 130 time frames per segment before resizing. The three representations are:

- **Mel spectrogram** (128 mel-scale frequency bins): captures perceptual timbre. Wolf-Monheim [3] demonstrates mel spectrograms substantially outperform other single representations for audio CNN classification, motivating their use as the primary branch.

- **STFT spectrogram** (1,025 linear-frequency bins): retains the full spectral content including high-frequency overtones that the mel scale compresses. It provides spectral detail diagnostic for percussive vs. tonal genres.
- **Chromagram** (12 pitch classes): projects spectral energy onto the Western chromatic scale, capturing harmonic and chord vocabulary. Classical and jazz exhibit complex, varied chord progressions; metal and blues are characterised by power chords and pentatonic patterns.

Each representation is converted to a power spectrogram (dB scale), normalised to $[0, 1]$, colourised with the plasma colormap, and resized to 128×128 . At training time, ImageNet normalisation (mean = $[0.485, 0.456, 0.406]$, std = $[0.229, 0.224, 0.225]$) is applied for compatibility with pretrained ResNet-18 weights [6]. The full mathematical derivation of each representation is given in Appendix A. Figure 1 shows all three for a representative segment.

2.3 Data Augmentation

To mitigate overfitting on the small training set (7,890 segments), four augmentations adapted from SpecAugment [7] are applied *offline* and exclusively to training segments, expanding the training set to 39,455 segments ($5\times$). Offline rather than online augmentation is used so that all three spectrogram modalities for the same segment receive identical augmentation masks, preserving cross-modal alignment. Augmentations are applied to raw spectrogram arrays before colormap application, so that the colormap nonlinearity does not interact with the masks:

- **Time masking**: 15% of time columns zeroed, simulating temporal occlusion.
- **Frequency masking**: 15% of frequency rows zeroed, applied independently per modality to respect their different frequency resolutions (1,025 bins for STFT vs. 12 for chroma).
- **Circular time shift**: the time axis is rolled by a random offset up to 10% of the frame width, simulating phase-shifted listening windows.
- **Gaussian noise**: additive noise with $\sigma = 0.02$, simulating mild recording artefacts.

Figure 2 illustrates each augmentation applied to a blues training segment. Full dataset counts after augmentation are provided in Appendix D.

2.4 Tabular Features

For Stage 3, the pre-extracted `features_3_sec.csv` file provides 57 numerical features per segment (MFCCs 1–20 with mean and variance, spectral centroid, spectral bandwidth, spectral rolloff, zero-crossing rate, RMS energy, and tempo). Exploratory correlation analysis (Appendix B) identifies three features with Pearson $|r| \geq 0.93$ against another retained feature: `rolloff_mean`, `spectral_bandwidth_mean`, and

`spectral_centroid_mean` are removed, yielding **54 features**. A `StandardScaler` is fit exclusively on training data and applied to val/test without re-fitting, preventing any leakage of test statistics into the feature normalisation.

3 Model Architecture

3.1 Backbone and Training Protocol

Figure 3 gives an overview of all three stages. All CNN branches use **ResNet-18** [4] as the feature extraction backbone, initialised with ImageNet pretrained weights. ResNet-18 is chosen for three reasons: (1) it is compact enough to train multiple parallel copies within GPU memory constraints; (2) Palanisamy et al. [8] demonstrate that ImageNet-pretrained ResNets are strong baselines for audio classification with spectrogram inputs; and (3) the GTZAN training set ($\approx 7,900$ segments before augmentation) is too small to train a deep network from random initialisation without severe overfitting.

The standard 1,000-class FC head is replaced with a task-specific classification head for each stage. The Global Average Pooling (GAP) layer that reduces the final $4 \times 4 \times 512$ feature map to a 512-d vector serves as the branch embedding output. Layer-by-layer dimensions are detailed in Appendix C.

Transfer learning and progressive unfreezing. Despite the domain shift from natural images to plasma-colourised spectrograms, early ResNet layers learn general edge and texture detectors that transfer well to time-frequency representations. To avoid destroying pretrained representations early in training, a progressive unfreezing schedule is adopted: layers 1–3 are frozen for the first 4 epochs (head-only training, $\text{lr} = 10^{-3}$), then **layer4** is unfrozen at epoch 5 with a reduced backbone $\text{lr} = 10^{-4}$, allowing high-level feature adaptation while lower layers remain stable.

Optimiser and regularisation. All models are trained with Adam [9] ($\beta_1 = 0.9$, $\beta_2 = 0.999$), a StepLR scheduler ($\gamma = 0.5$, step 10 epochs), and early stopping on validation loss (patience 7). At every epoch, model weights achieving the best validation accuracy are checkpointed and restored at the end of training. Full hyperparameters are listed in Appendix C.

3.2 Stage 1: Single-Branch Mel Baseline

Stage 1 establishes a single-modality baseline using mel spectrograms as the sole CNN input. The classification head appended after the 512-d GAP embedding is: $\text{FC}(512 \rightarrow 256) + \text{ReLU}$, $\text{Dropout}(0.4)$, $\text{FC}(256 \rightarrow 10)$. The intermediate 256-d hidden layer introduces a non-linear bottleneck; dropout with $p = 0.4$ provides regularisation appropriate for the dataset size. Head-only trainable parameters: $(512 \times 256 + 256) + (256 \times 10 + 10) = 133,898$.

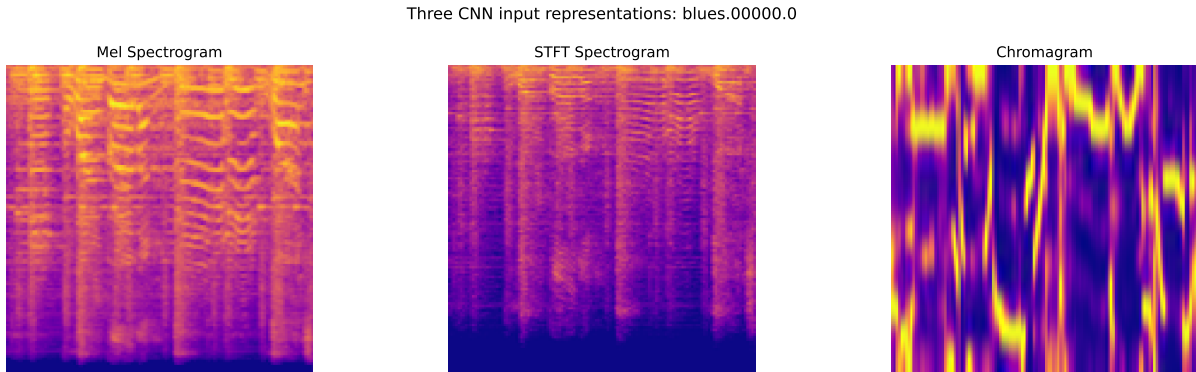


Figure 1: The three CNN input representations for the same 3-second blues segment. Left to right: mel spectrogram (128 mel bins), STFT spectrogram (linear frequency), chromagram (12 pitch classes). Each captures complementary acoustic information.

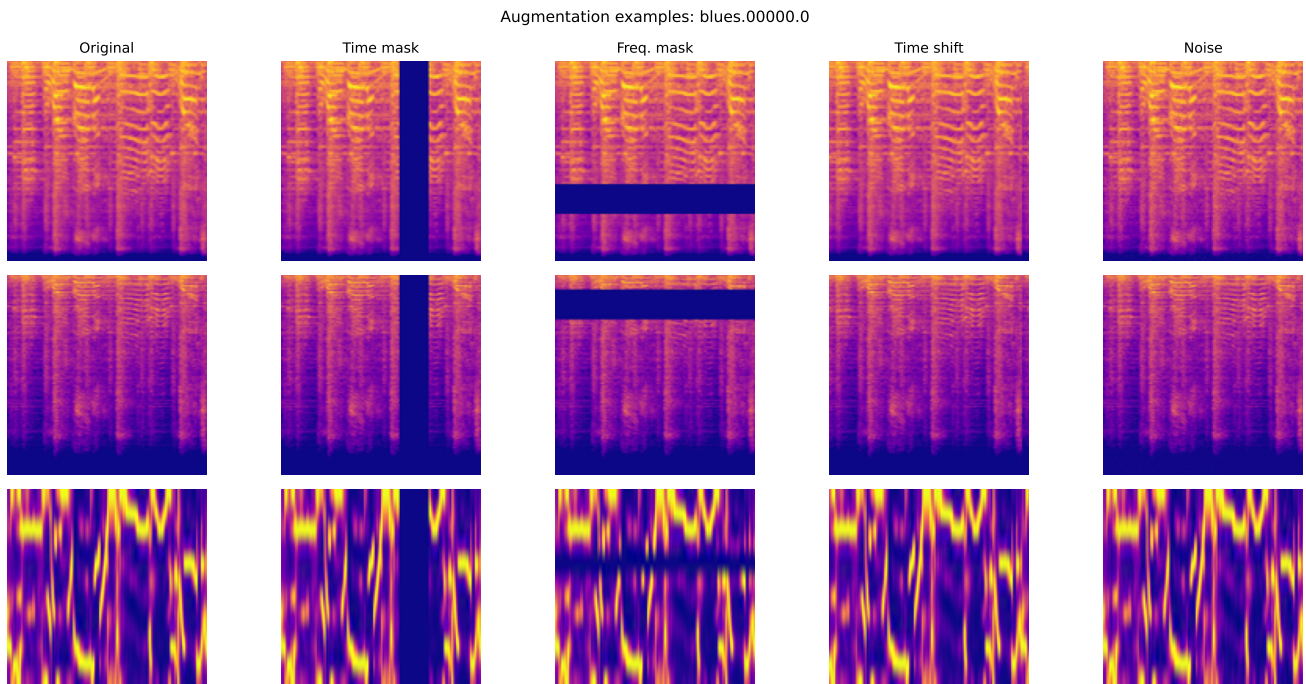


Figure 2: Effect of the four offline augmentations on a blues training segment. Rows: mel, STFT, chromagram. Columns: original, time mask, frequency mask, time shift, Gaussian noise.

3.3 Stage 2: Three-Branch Fusion

Stage 2 extends Stage 1 by processing all three spectral representations in parallel. Three independent ResNet-18 branches (mel, STFT, chroma) each produce 512-d embeddings via GAP, which are concatenated into a 1,536-d joint representation and passed through a shared fusion head: FC(1536→512)+ReLU, Dropout(0.4), FC(512→256)+ReLU, Dropout(0.3), FC(256→10).

The two-stage compression (1,536→512→256) allows cross-modal interactions to be learned at multiple levels of abstraction. The three modalities are chosen to provide **complementary information**: mel captures perceptual timbre; STFT retains high-frequency spectral detail that mel’s logarithmic compression discards; and chroma encodes harmonic/chord vocabulary largely absent from mel spectrograms [3, 5]. The batch size is reduced to 32 (from 64 in Stage 1) to accommodate

the tripled memory footprint of loading three spectrograms per sample. Each branch’s `layer4` is unfrozen simultaneously at epoch 5.

3.4 Stage 3: CNN Fusion with Tabular Branch

Stage 3 adds a fourth branch: a two-layer MLP (FC(54→128)+ReLU, Dropout(0.4)) that maps the 54 standardised tabular features to a 128-d embedding. The 128-d tabular embedding is *directly concatenated* with the 1,536-d CNN embedding, without first compressing the CNN representation. This design allows the joint FC head (FC(1664→512)+ReLU, Dropout(0.4), FC(512→256)+ReLU, Dropout(0.3), FC(256→10)) to learn the optimal compression of both modalities jointly, enabling it to discover interactions between CNN and tabular features rather than forcing them into separate

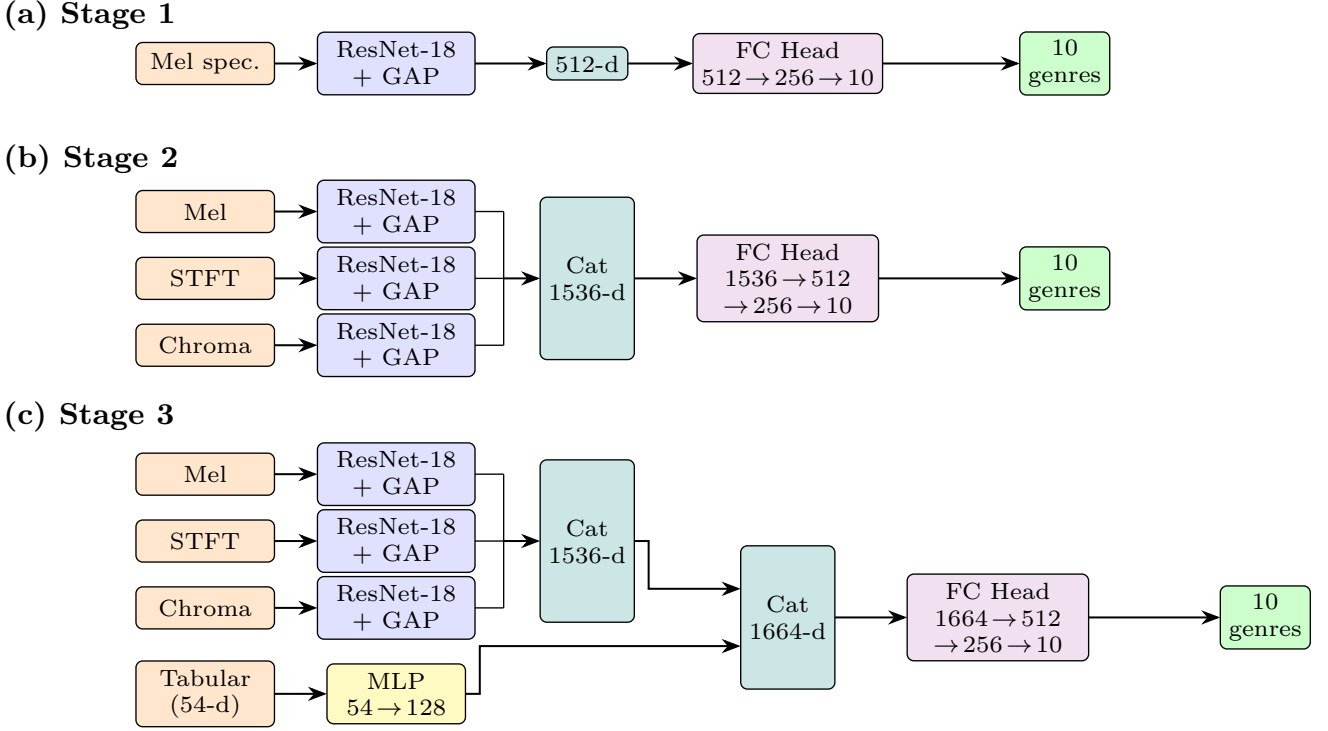


Figure 3: Architecture overview of the three model stages. (a) Stage 1: single ResNet-18 branch on mel spectrograms. (b) Stage 2: three parallel branches (mel, STFT, chroma) concatenated into a 1536-d joint embedding before a shared FC head. (c) Stage 3: same three CNN branches plus a tabular MLP branch; CNN embeddings are first concatenated (1536-d), then concatenated with the 128-d tabular embedding (1664-d total) before the FC head. Colours: orange = input, blue = ResNet-18+GAP, yellow = tabular MLP, teal = concatenation, purple = FC head, green = output.

bottlenecks.

The hypothesis is that hand-crafted MIR features accumulated through decades of domain expertise [1] encode complementary global statistics (tempo, MFCC cepstral coefficients, zero-crossing rate, RMS) that convolutional operations on local spectrogram patches may not fully capture. Projecting first to 128 dimensions (slightly larger than the 54-d input) allows the MLP to learn non-linear feature interactions before fusion.

4 Evaluation

4.1 Experimental Setup

All three stages are evaluated on the held-out test split of 990 three-second segments (99 tracks, ≈ 10 per genre), which was never used during training or hyperparameter selection. The primary metric is top-1 accuracy: the fraction of test segments for which the model’s argmax prediction matches the ground-truth label. For each stage, the weights restored at the end of training are the best-val-accuracy checkpoint.

4.2 Results

Table 1 shows that each additional stage yields a consistent improvement. Stage 1→2 gains +2.83 pp by adding STFT and chroma branches, confirming that the two additional modalities provide complementary discriminative signal beyond mel alone. Stage 2→3 gains a further

+2.32 pp by adding the tabular MLP branch, confirming that hand-crafted MIR statistics encode information not captured by the CNN branches. The consistent monotonic improvement across stages validates each architectural hypothesis.

Table 1: Test-set accuracy for each model stage.

Stage	Architecture	Test Acc.
Stage 1	Single-branch ResNet-18 (mel)	75.96%
Stage 2	Three-branch CNN fusion	78.79%
Stage 3	CNN fusion + tabular MLP	81.11%

4.3 Per-Class Analysis

Table 2 reports the Stage 3 per-genre classification report and Figure 4 shows the full confusion matrix. **Classical** (F1 = 0.94) and **jazz** (F1 = 0.90) are the best-performing genres. Classical benefits from its highly distinctive orchestral timbres and lack of percussion, making it a near-isolated cluster in feature space (visible in the PCA plot, Appendix B). Jazz scores well due to its complex harmonic vocabulary, which the chroma branch captures effectively.

Pop and **rock** are the most challenging (F1 = 0.67 each). Rock is frequently misclassified as blues or country, which share guitar-dominant timbres and similar rhythmic structures. Pop is confused with country and disco, reflecting overlapping production styles and rhythmic templates across these commercially adjacent genres. **Disco**

(F1 = 0.71) also suffers from its rhythmic overlap with hip-hop and its stylistic closeness to pop. This is unsurprising, considering that even human annotators disagree at genre boundaries, and the 30-second clips further reduce the available acoustic context.

Table 2: Per-genre classification report, Stage 3 (test set, $n = 990$).

Genre	Prec.	Recall	F1
Blues	0.81	0.91	0.85
Classical	0.93	0.94	0.94
Country	0.79	0.82	0.80
Disco	0.72	0.70	0.71
Hip-hop	0.89	0.78	0.83
Jazz	0.95	0.86	0.90
Metal	0.82	0.91	0.86
Pop	0.66	0.69	0.67
Reggae	0.87	0.88	0.88
Rock	0.71	0.63	0.67
Macro avg	0.81	0.81	0.81

4.4 Comparison with Baseline

The reference baseline is the most upvoted Kaggle notebook by Olteanu [10] (the author who compiled the GTZAN dataset on Kaggle), which trains classical ML classifiers on tabular features. Table 3 compares all baselines against the proposed stages. XGBoost achieves a headline 90.22%, outperforming our best Stage 3 model by 9.11 pp.

Table 3: Tabular ML baselines [10] vs. proposed CNN stages.

Model	Input	Acc.
Naive Bayes	Tabular	51.95%
SGD Classifier	Tabular	65.53%
Decision Tree	Tabular	64.00%
Logistic Regression	Tabular	69.77%
MLP (shallow)	Tabular	67.73%
SVM (RBF)	Tabular	75.41%
KNN	Tabular	80.58%
XGBoost (RF mode)	Tabular	74.88%
XGBoost	Tabular	90.22%
Stage 1 (ours)	Mel spectrogram	75.96%
Stage 2 (ours)	Mel+STFT+chroma	78.79%
Stage 3 (ours)	Images+tabular	81.11%

4.5 Discussion

Data leakage in the baseline. The 9.11 pp gap between Stage 3 and XGBoost is partly a methodological artefact of two data leakages in the baseline. **Leakage 1:** the baseline applies `train_test_split` directly to the 9,990-row segment dataframe, so segments from the same track appear in both train and test. Because segments from the same 30-second recording are acoustically near-identical, the model effectively memorises tracks rather

than learning genre-discriminative features [2]. **Leakage 2:** `MinMaxScaler.fit_transform` is called on the full 9,990-row dataframe before splitting, contaminating the scaler with test-set statistics. Both leakages inflate the reported accuracy. Our strict track-level protocol produces a harder but methodologically sound evaluation.

Inherent advantages of the tabular baseline. Beyond leakage, XGBoost benefits from two structural advantages. First, the MIR tabular features are *expert-designed* to be genre-discriminative (tempo, MFCCs, spectral shape) and directly encode domain knowledge accumulated over decades. Second, the GTZAN dataset’s small size (988 tracks) structurally favours low-dimensional tree-based models: gradient-boosted trees require far fewer samples to generalise than a three-branch CNN with millions of parameters, even with transfer learning and aggressive augmentation.

Directions for improvement. Several directions could close the remaining gap. **Test-time aggregation** (majority or soft vote across the 10 segments of each track) would boost accuracy at inference time with no additional training cost, since track-level predictions are more stable than individual 3-second segment predictions. **Audio-domain pretraining** via VGGish [11] or PANNs [12] would replace ImageNet priors with representations pre-trained on millions of audio clips, better suited to spectro-temporal patterns. **Systematic hyperparameter search** (learning rate, dropout rates, FC widths, MLP depth, augmentation strengths) was not performed due to compute constraints and could yield meaningful gains across all stages.

5 Conclusion

This work demonstrates that progressively richer fusion of acoustic representations yields consistent accuracy gains on GTZAN: Stage 1 (single-branch mel, 75.96%) → Stage 2 (three-branch fusion, 78.79%) → Stage 3 (CNN+tabular, 81.11%). Each architectural addition validates its underlying hypothesis: STFT and chroma branches provide complementary spectral and harmonic information, and hand-crafted MIR statistics provide global segment-level cues not fully captured by local convolutional operations.

The 9.11 pp gap to the XGBoost tabular baseline (90.22%) is partly a methodological artefact: the baseline uses segment-level splitting and pre-split feature scaling, both of which inflate its reported accuracy. On a fair evaluation, the advantage of expert-designed tabular features over ImageNet-pretrained CNNs on a 988-track dataset is expected and reflects the structural advantage of gradient-boosted trees on small, low-dimensional feature matrices.

Closing the gap fully would likely require moving beyond GTZAN’s constraints: audio-domain pretraining (VGGish [11], PANNs [12]), track-level inference aggregation, or end-to-end waveform learning. The staged

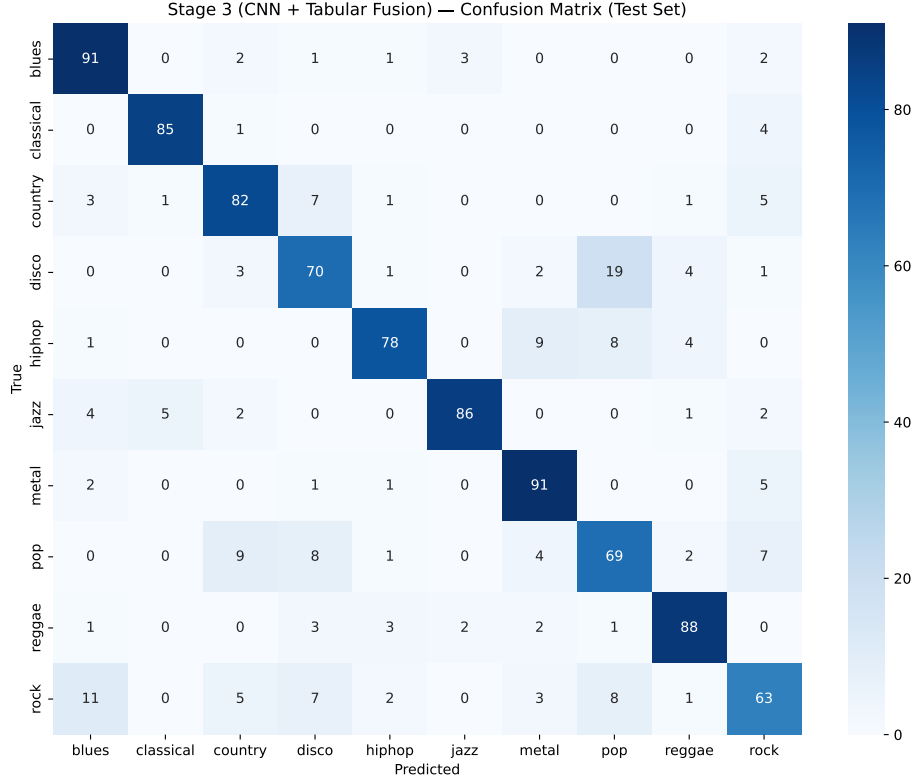


Figure 4: Confusion matrix for the Stage 3 model on the 990-segment test set (raw segment counts). Rows are true labels; columns are predicted labels.

fusion architecture developed here nonetheless provides a reusable blueprint for multimodal audio classification under data-scarce conditions, with each modality’s contribution cleanly isolated and empirically validated.

References

- [1] G. Tzanetakis and P. Cook, “Musical genre classification of audio signals,” *IEEE Transactions on Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, Jul. 2002.
- [2] B. L. Sturm, “The GTZAN dataset: Its contents, its faults, their effects on evaluation, and its future use,” 2013.
- [3] F. Wolf-Monheim, “Spectral and rhythm features for audio classification with deep convolutional neural networks,” 2024.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [5] Y. Zhang and T. Li, “Music genre classification with parallel convolutional neural networks and capuchin search algorithm,” *Scientific Reports*, vol. 15, p. 9580, 2025.
- [6] PyTorch Contributors, “Models and pre-trained weights — TorchVision 0.9 documentation,” 2021. [Online]. Available: <https://docs.pytorch.org/vision/0.9/models.html>
- [7] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, “SpecAugment: A simple data augmentation method for automatic speech recognition,” in *Proc. Interspeech*, 2019, pp. 2613–2617.
- [8] K. Palanisamy, D. Singhania, and A. Yao, “Rethinking CNN models for audio classification,” 2020.
- [9] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. International Conference on Learning Representations (ICLR)*, 2015.
- [10] A. Olteanu, “Work w/ Audio data: Visualise, classify, recommend,” Kaggle Notebook, 2020. [Online]. Available: <https://www.kaggle.com/code/andradaolteanu/work-w-audio-data-visualisation-and-classification>
- [11] S. Hershey, S. Chaudhuri, D. P. W. Ellis, J. F. Gemmeke, A. Jansen, R. C. Moore, M. Plakal, D. Platt, R. A. Saurous, B. Seybold, M. Slaney, R. Weiss, and K. Wilson, “CNN architectures for large-scale audio classification,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 131–135.
- [12] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, “PANNs: Large-scale pretrained audio neural networks for audio pattern recognition,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2880–2894, 2020.
- [13] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Pearson Prentice Hall, 2010.

- [14] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [15] J. B. Allen and L. R. Rabiner, “A unified approach to short-time Fourier analysis and synthesis,” *Proceedings of the IEEE*, vol. 65, no. 11, pp. 1558–1564, 1977.
- [16] D. O’Shaughnessy, *Speech Communication: Human and Machine*. Addison-Wesley, 1987.
- [17] S. B. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.
- [18] M. Müller, *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Springer, 2015.

A Mathematical Foundations of Acoustic Representations

Discrete Fourier Transform. A digital audio signal is a discrete sequence of air pressure measurements sampled at rate f_s . The **Discrete Fourier Transform (DFT)** decomposes a finite-length signal $x[n]$ of length N into its constituent complex sinusoids [13]:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, \dots, N-1$$

where $X[k]$ is the complex amplitude of frequency $f_k = kf_s/N$. The magnitude $|X[k]|$ indicates how strongly that frequency is present. The FFT computes this in $O(N \log N)$ [14].

Short-Time Fourier Transform. Music is non-stationary, so the DFT of an entire track discards temporal structure. The **STFT** computes the DFT over short, overlapping windows [15]:

$$\mathbf{S}[m, k] = \sum_{n=0}^{N_{\text{fft}}-1} x[n + mH] \cdot w[n] \cdot e^{-j2\pi kn/N_{\text{fft}}}$$

where m is the frame index, H is the hop length, and $w[n]$ is a Hann window that reduces spectral leakage. The power spectrogram $|\mathbf{S}[m, k]|^2$ is converted to dB:

$$\mathbf{S}_{\text{dB}}[m, k] = 10 \log_{10}(|\mathbf{S}[m, k]|^2) - 10 \log_{10} \left(\max_{m,k} |\mathbf{S}[m, k]|^2 \right)$$

compressing the dynamic range to roughly $[-80, 0]$ dB. Parameters used: $N_{\text{fft}} = 2048$, $H = 512$, $f_s = 22,050$ Hz, yielding ≈ 130 time frames and 1,025 frequency bins per 3-second segment.

Mel Spectrogram. Human pitch perception is logarithmic in frequency. The *mel scale* models this [16]:

$$m(f) = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right)$$

A bank of $M_{\text{mel}} = 128$ triangular filters, uniformly spaced on the mel axis, is applied to the STFT power spectrum [17]:

$$\mathbf{M}[m, l] = \sum_k \mathbf{H}_l[k] \cdot |\mathbf{S}[m, k]|^2, \quad l = 1, \dots, 128$$

giving a 128×130 representation that captures perceptual timbre. Wolf-Monheim [3] show mel spectrograms substantially outperform other single representations for audio CNN classification.

Chromagram. A **chromagram** projects the spectrum onto the 12 pitch classes of the Western chromatic scale by summing energy across all octaves [18]:

$$\mathbf{C}[m, p] = \sum_{k: \text{pitch}(f_k)=p} |\mathbf{S}[m, k]|^2, \quad p \in \{0, \dots, 11\}$$

The resulting $12 \times M$ matrix captures harmonic and tonal structure, strongly discriminating genres by chord vocabulary (e.g., jazz complex chords vs. metal power chords).

Table 4: Summary of the three CNN input representations.

Representation	Axes	Raw shape	Captures
STFT Spectrogram	Time $\times$ Freq. (linear)	1025×130	Full spectral content
Mel Spectrogram	Time $\times$ Freq. (mel)	128×130	Perceptual timbre
Chromagram	Time $\times$ Pitch class	12×130	Harmonic/tonal content

B Exploratory Data Analysis

Feature Distributions. Kernel density estimates of six representative tabular features (Figure 5) confirm that no single feature cleanly separates all 10 genres. Tempo discriminates fast-paced genres (metal, disco) from slower ones (blues, classical); spectral centroid is elevated for high-energy genres (pop, metal); zero-crossing rate is highest for percussive and distorted genres.

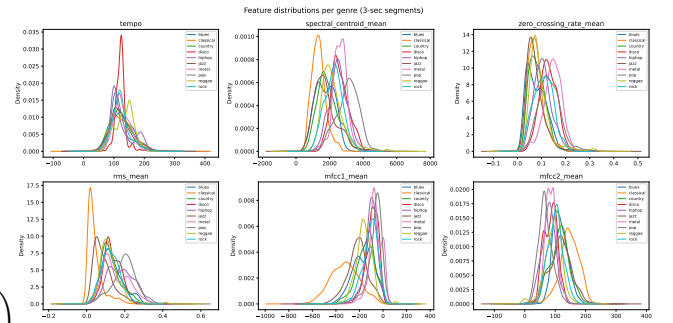


Figure 5: Kernel density estimates of six representative features across 10 genres (3-second segments). Overlapping distributions indicate no single feature is sufficient for reliable genre separation.

Feature Correlation Analysis. Pairwise Pearson correlations among the 57 numerical features reveal four highly correlated pairs ($|r| \geq 0.90$), listed in Table 5. `rolloff_mean` appears in three of the four pairs; together with `spectral_bandwidth_mean` and `spectral_`

centroid_mean it is dropped, retaining mfcc2_mean as it is empirically the most discriminative feature group [1]. This reduces feature dimensionality from 57 to 54.

Table 5: Highly correlated feature pairs ($|r| \geq 0.90$).

Feature 1	Feature 2	Corr.
spectral_centroid_mean	rolloff_mean	+0.974
spectral_bandwidth_mean	rolloff_mean	+0.951
spectral_centroid_mean	mfcc2_mean	-0.931
rolloff_mean	mfcc2_mean	-0.923

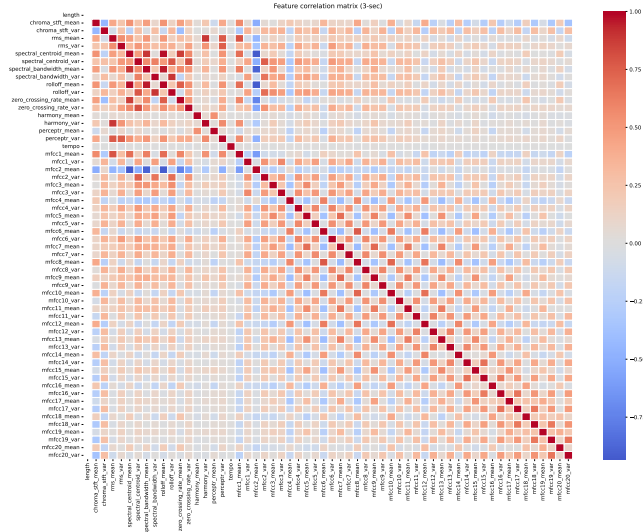


Figure 6: Pairwise Pearson correlation matrix of the 57 numerical features.

PCA Visualisation and Outlier Detection. PCA on the standardised 57-feature matrix yields a 2D projection explaining $\approx 33.6\%$ of variance. The projection (Figure 7) shows heavy genre entanglement, motivating non-linear models. Outliers are segments whose Euclidean distance from their genre centroid in PCA space exceeds 3σ ; tracks with ≥ 8 outlier segments out of 10 are flagged as track-level anomalies and removed during cleaning.

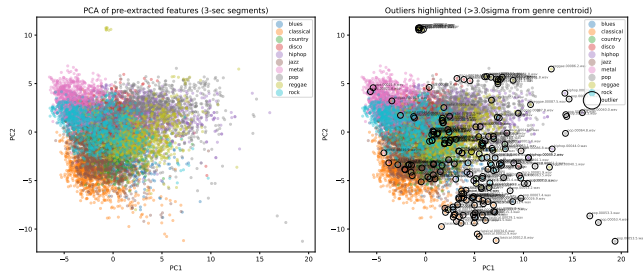


Figure 7: 2D PCA projection (standardised features). Left: genre clusters. Right: outlier segments ($> 3\sigma$ from genre centroid) highlighted. Classical forms the most compact cluster; rock, country, and blues overlap heavily.

C Detailed Model Architectures and Hyperparameters

Table 6: ResNet-18 layer-by-layer output dimensions for a $128 \times 128 \times 3$ input.

Stage	Output shape	Description
Input	$128 \times 128 \times 3$	RGB spectrogram
Stem (Conv1+MaxPool)	$32 \times 32 \times 64$	7×7 conv stride 2; 3×3 maxpool stride 2
Layer 1	$32 \times 32 \times 64$	$2 \times$ BasicBlock, 64 ch
Layer 2	$16 \times 16 \times 128$	$2 \times$ BasicBlock, 128 ch, stride 2
Layer 3	$8 \times 8 \times 256$	$2 \times$ BasicBlock, 256 ch, stride 2
Layer 4	$4 \times 4 \times 512$	$2 \times$ BasicBlock, 512 ch, stride 2
GAP	512	Global Average Pooling

Table 7: Training hyperparameters, common across all stages unless noted.

Hyperparameter	Value	Notes
Optimiser	Adam	$\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$ [9]
Loss function	Cross-entropy	Standard multi-class
Head LR	10^{-3}	Epochs 1+
Backbone LR (layer4)	10^{-4}	From epoch 5
LR scheduler	StepLR	Step 10, $\gamma = 0.5$
Unfreeze epoch	5	layer4 added at epoch 5
Max epochs	30	Hard limit
Early stopping patience	7 epochs	Monitors val. loss
Best-weight criterion	Max val. acc	Checkpoint restored at end
Batch size (Stage 1)	64	Single spectrogram/sample
Batch size (Stages 2 & 3)	32	Three spectrograms/sample
Dropout (first FC)	0.4	All heads
Dropout (second FC)	0.3	Stages 2 & 3 only
Random seed	42	torch and numpy

Table 8: Stage 1 architecture (single-branch mel baseline).

Layer	Output dim	Notes
Input (mel spectrogram)	$128 \times 128 \times 3$	Plasma-colourised, ImageNet-normalised
ResNet-18 + GAP	512	Pretrained; layers 1-3 frozen
FC(512 $\rightarrow$ 256) + ReLU	256	
Dropout(0.4)	256	
FC(256 $\rightarrow$ 10)	10	Genre logits

Table 9: Stage 2 architecture (three-branch CNN fusion).

Layer	Output dim	Notes
Mel / STFT / Chroma ResNet-18 + GAP	3×512	Pretrained; layers 1-3 frozen
Concatenation	1536	512×3 joint embedding
FC(1536 $\rightarrow$ 512) + ReLU	512	
Dropout(0.4)	512	
FC(512 $\rightarrow$ 256) + ReLU	256	
Dropout(0.3)	256	
FC(256 $\rightarrow$ 10)	10	Genre logits

Table 10: Stage 3 architecture (CNN fusion + tabular MLP branch).

Layer	Output dim	Notes
Mel / STFT / Chroma ResNet-18 + GAP	3×512	
Concatenation (CNN)	1536	
Tabular input	54	Standardised
FC(54 $\rightarrow$ 128) + ReLU	128	
Dropout(0.4)	128	
Concatenation (CNN + tabular)	1664	$1536 + 128$
FC(1664 $\rightarrow$ 512) + ReLU	512	
Dropout(0.4)	512	
FC(512 $\rightarrow$ 256) + ReLU	256	
Dropout(0.3)	256	
FC(256 $\rightarrow$ 10)	10	Genre logits

D Dataset Size at Each Processing Stage

Table 11: Segment counts after cleaning and augmentation.

Split	Orig.	Aug.	Total segs	Images
Train	7,891	31,564	39,455	118,365
Val	990	0	990	2,970
Test	990	0	990	2,970
Total	9,871	31,564	41,435	124,305